UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 8: Extending Libraries & Generators

Wednesday Apr 1, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 2-3 to solve a few exercises
    - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions
- Solutions to exercises will be reviewed as a class
- Attendance taken at the end

Today's Goals

- Practice with Extending Libraries
- Practice with Generators

Reminders

- HW 7 is due tonight (Wed 04/01) at 11:59pm.
  - Come to office hours for help!
- If you need to miss lab and have a valid reason according to the syllabus (medical, other personal) please fill out the questionnaire on Canvas before the start time of your lab.
  - Waking up late, bus was late are NOT valid reasons to miss lab.

Extending Libraries

Two main ways to extend library code in OOP setting:

- **Inheritance**
  - Extend a class using `extends`. Easy to make new data type with modified versions of superclass methods.
- **Composition**
  - Wrap a class by having an property of the library type, then defer responsibilities to that property

Inheritance Example

```
class Phone() {
    call(phoneNum: number): void { ... }
}

class Smartphone extends Phone {
    private getNumberFromContactName(contact: string): number { ... }

    callByContact(contact: string): void {
        super.call(getNumberFromContactName(contact));
    }
    openInternetBrowser() { ... }
    // ...
}
```

Composition Example

```
class Phone() {
    call(phoneNum: number): void { ... }
}

class Smartphone {
    private phoneApp: Phone;
    private getNumberFromContactName(contact: string): number { ... }

    call(phoneNum: number): void { this.phoneApp.call(phoneNum); }
    callByContact(contact: string): void {
        phoneApp.call(getNumberFromContactName(contact));
    }
    openInternetBrowser() { ... }
    // ...
}
```

Exercise 1: Extending Libraries

Given a Polygon class, extend the class to CenteredPolygon which has a method to compute its center.

The center is the average of the vertex coordinates.

```
class CenteredPolygon extends Polygon {.......}
```

Generators Review

A class that extends Iterator must implement:

- `next()`: returns next item, and advances internal state for future calls to `next()`

Handling internal state is not always obvious how to do.

Generators can be more concise and easier to work with.

- They implement IterableIterator: calling `next()` yields an `IteratorResult`.
- The main difference is how generators are defined and created.

Generators Review

Generators need a **generator** function, defined with `function*`, which uses `yield` instead of `return`.

Successive calls to `next()` return the next yielded item.

```
// declaring a generator function
function* generator() {
  yield 1;
  yield 2;
  yield 3;
}
```

The yielded item is an `IteratorResult` (not `number`). The `yield` keyword automatically performs the conversion.

```
const gen = generator(); // create generator
console.log(gen.next().value); // prints 1
console.log(gen.next().value); // prints 2
console.log(gen.next().value); // prints 3
```

Exercise 2: Generators

Implement a generator (chainGenerator) for the ChainIterator problem. It should behave the same way as ChainIterator from last week's lab:

- Takes in an array of `Iterable`s.
- Yield items from the first iterable until it runs out, then the second, etc.
- Repeat for each array in `iterables`.
- Assume `iterables` is not empty.

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  // TODO: Implement this generator
}
```